

Repository Pattern with Async SQLAlchemy

This is one of the most common patterns in production FastAPI projects.

The idea is simple:

The API should not directly write SQLAlchemy queries. Instead, it should call a **Repository**, and the repository handles all database operations. This separates business logic from data access and makes the code easier to test and maintain.

Your API is doing **two jobs**:

1. Handling the HTTP request.
2. Talking to the database.

This works, but if you have 100 endpoints, you'll repeat the same SQLAlchemy code everywhere.

With Repository Pattern

Now the endpoint becomes:

```
@app.post("/users")
async def create_user(
    user: UserCreate,
    db: AsyncSession = Depends(get_db)
):
    repository = UserRepository(db)
    return await repository.create(user)
```

Now:

- API → Handles HTTP
- Repository → Handles database

This separation is a core goal of the Repository pattern.

Project Structure

```
project/  
  
database.py  
models.py  
schemas.py  
  
repositories/  
    user_repository.py  
  
main.py
```

Step 1 Database

```
# database.py  
  
from sqlalchemy.ext.asyncio import (  
    create_async_engine,  
    AsyncSession,  
    async_sessionmaker  
)  
  
engine = create_async_engine(  
    "sqlite+aiosqlite:///users.db"  
)  
  
AsyncSessionLocal = async_sessionmaker(engine,  
    class_=AsyncSession,  
    expire_on_commit=False  
)  
  
async def get_db():  
  
    async with AsyncSessionLocal() as session:  
        yield session
```

Nothing changes.

Step 2 Model

```
from sqlalchemy import Column, Integer, String  
from sqlalchemy.orm import declarative_base
```

```
Base=declarative_base()

class User(Base):

    __tablename__="users"

    id=Column(Integer,primary_key=True)
    name=Column(String)
    email=Column(String)
```

Step 3 Schema

```
from pydantic import BaseModel

class UserCreate(BaseModel):

    name:str
    email:str
```

Step 4 Create Repository

Create

```
repositories/user_repository.py
```

Constructor

```
from sqlalchemy.ext.asyncio import AsyncSession

class UserRepository:

    def __init__(self,db:AsyncSession):
        self.db=db
```

What happens here?

When you do

```
repo=UserRepository(db)
```

Python executes

```
def __init__(self,db):  
  
    self.db=db
```

So

```
self.db
```

stores the AsyncSession.

Think

```
UserRepository  
  
↓  
  
self.db  
  
↓  
  
AsyncSession
```

Create User

```
from models import User  
  
class UserRepository:  
  
    def __init__(self,db):  
  
        self.db=db  
  
    async def create(self,data):  
  
        user=User(  
            name=data.name,  
            email=data.email  
        )
```

```
self.db.add(user)

await self.db.commit()

await self.db.refresh(user)

return user
```

Notice

There is **no FastAPI code** here.

Only SQLAlchemy.

Read All Users

```
from sqlalchemy import select

async def get_all(self):

    result=await self.db.execute(

        select(User))

    return result.scalars().all()
```

Get User By ID

```
async def get_by_id(self,user_id:int):

    result=await self.db.execute(select(User).where(User.id==user_id))

    return result.scalars().first()
```

Update User

```
async def update(self,user_id:int,data):

    result=await self.db.execute(select(User).where(User.id==user_id))

    user=result.scalars().first()
```

```
        if not user:
            return None

        user.name=data.name
        user.email=data.email

        await self.db.commit()
        await self.db.refresh(user)

        return user
```

Delete User

```
async def delete(self, user_id: int):

    result = await self.db.execute(select(User).where(User.id == user_id))

    user = result.scalars().first()

    if not user:

        return False

    await self.db.delete(user)

    await self.db.commit()

    return True
```

API Layer

Now your API becomes very clean.

Create

```
@app.post("/users")
async def create_user(

    user: UserCreate,

    db: AsyncSession = Depends(get_db)

):
```

```
repository=UserRepository(db)

return await repository.create(user)
```

Get All

```
@app.get("/users")
async def get_users(

    db:AsyncSession=Depends(get_db)

):

    repository=UserRepository(db)

    return await repository.get_all()
```

Get One

```
@app.get("/users/{id}")
async def get_user(

    id:int,

    db:AsyncSession=Depends(get_db)

):

    repository=UserRepository(db)

    return await repository.get_by_id(id)
```

Update

```
@app.put("/users/{id}")
async def update_user(

    id:int,

    data:UserCreate,

    db:AsyncSession=Depends(get_db)
```

```
):  
  
repository=UserRepository(db)  
  
return await repository.update(id,data)
```

Delete

```
@app.delete("/users/{id}")  
async def delete_user(  
  
    id:int,  
  
    db:AsyncSession=Depends(get_db)  
  
):  
  
    repository=UserRepository(db)  
  
    return await repository.delete(id)
```

Complete Flow

```
Client  
|  
▼  
FastAPI Endpoint  
|  
▼  
Depends(get_db)  
|  
▼  
AsyncSession  
|  
▼  
UserRepository(db)  
|  
▼  
Repository Method  
|  
▼  
SQLAlchemy Query  
|
```


Benefits include:

- The API layer focuses on HTTP requests and responses.
- Database logic is centralized in one place.
- The same repository can be reused by multiple services.
- Unit testing becomes easier because repositories can be mocked.

Interview Answer

Question: Why do we use the Repository Pattern?

Answer:

The Repository Pattern abstracts database operations into a dedicated class. Instead of writing SQLAlchemy queries inside every FastAPI endpoint, the endpoints call repository methods such as `create()`, `get_by_id()`, or `delete()`. This separation of concerns makes the codebase cleaner, easier to maintain, easier to test, and reusable across different parts of the application.